UNIVERSIDAD TECNOLÓGICA DEL PERÚ

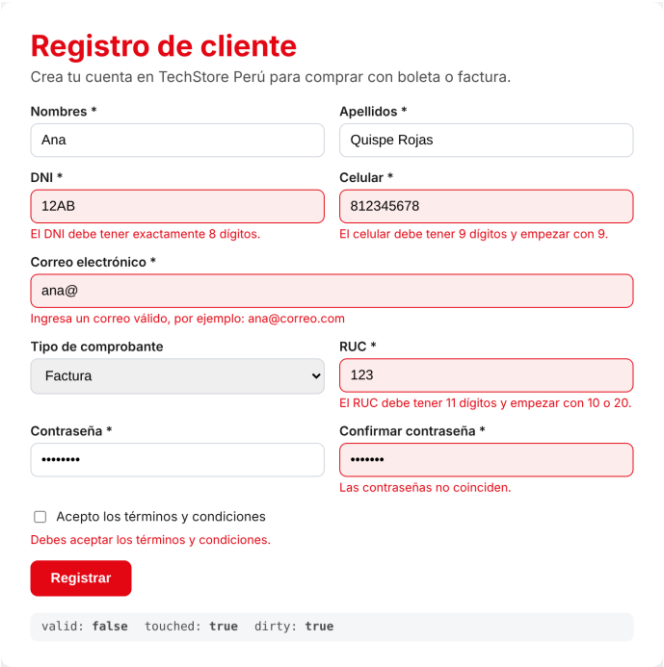
Desarrollo Web Integrado (100000ST61) · Unidad 3: Integración de proyectos web

TALLER DE LABORATORIO N.º 13

Procesamiento de formularios, manejo de errores y validación de datos

Caso TechStore Perú SAC: registro de clientes con Angular 22 (Reactive Forms) y Spring Boot 4 (Bean Validation y @RestControllerAdvice)

Dato	Detalle
Curso	Desarrollo Web Integrado (100000ST61)
Semana / Sesión	Semana 13 · Sesión 13
Tema del syllabus	Procesamiento de formularios. Manejo de errores y validación de datos.
Duración sugerida	90 minutos (Parte A: 30 min · Parte B: 35 min · Parte C: 25 min)
Modalidad	Individual o en parejas
Docente	Mag. Ing. Marco Antonio Flores Torres
Estudiante(s)	_____
Fecha	____ / ____ / 2026



Lo que construirás: formulario de registro de TechStore con validaciones y mensajes de error

1. Logro del taller

Al finalizar el taller, el estudiante **implementa** un formulario reactivo en Angular con validación de datos y manejo de errores en el cliente y en el servidor (Bean Validation y `@RestControllerAdvice` en Spring Boot), aplicando buenas prácticas de experiencia de usuario y seguridad en el caso TechStore Perú.

Lo que practicarás

- Crear un formulario con **Reactive Forms**: `FormGroup`, `FormControl`, `NonNullableFormBuilder` y `Validators`.
- Escribir **validadores personalizados** para datos peruanos (DNI, celular y RUC) y un **validador cruzado** (contraseña y confirmación).
- Aplicar **validación condicional**: el RUC solo es obligatorio cuando el cliente pide factura.
- Validar en el servidor con **Bean Validation** (`@NotBlank`, `@Pattern`, `@Email`, `@AssertTrue`) y `@Valid`.
- Centralizar el **manejo de errores** con `@RestControllerAdvice` y responder un JSON uniforme (400, 404, 409 y 500).
- Mostrar en cada campo del formulario los **errores que devuelve el servidor**.

2. Escenario: TechStore Perú SAC

TechStore Perú SAC es la tienda de tecnología que venimos construyendo en el curso. El área comercial detectó que muchos clientes se registran con datos incorrectos: DNI con letras, celulares incompletos y facturas pedidas sin RUC. Por eso, la SUNAT les ha rechazado comprobantes y los repartidores no logran comunicarse con los clientes.

Te piden construir el **registro de clientes** con estas reglas de negocio, que deben cumplirse **en el formulario (Angular) y también en la API (Spring Boot)**:

Campo	Regla	Mensaje para el usuario
Nombres / Apellidos	Obligatorio, entre 2 y 50 caracteres	Este campo es obligatorio / Debe tener al menos 2 caracteres
DNI	Obligatorio, exactamente 8 dígitos y no repetido	El DNI debe tener exactamente 8 dígitos
Correo electrónico	Obligatorio y con formato válido	Ingresa un correo válido, por ejemplo: ana@correo.com
Celular	Obligatorio, 9 dígitos y empieza con 9	El celular debe tener 9 dígitos y empezar con 9
Tipo de comprobante	BOLETA o FACTURA	—
RUC	Solo si es FACTURA: 11 dígitos que empiezan con 10 o 20	El RUC debe tener 11 dígitos y empezar con 10 o 20
Contraseña	Obligatoria, mínimo 8 caracteres	Debe tener al menos 8 caracteres
Confirmar contraseña	Igual a la contraseña	Las contraseñas no coinciden
Términos y condiciones	Debe estar marcado	Debes aceptar los términos y condiciones

⚠**IMPORTANTE:** El front-end valida para **ayudar** al usuario; el back-end valida para **proteger** el sistema. Cualquiera puede enviar un JSON a la API con Postman sin pasar por Angular, así que la API nunca debe confiar en el cliente.

3. Requisitos previos

Herramienta	Versión sugerida	¿Para qué?
JDK	21	Compilar y ejecutar la API Spring Boot
IntelliJ IDEA	Community o Ultimate (2025.x o superior)	Editar y ejecutar ambos proyectos
Node.js	LTS vigente (22 o 24)	Ejecutar Angular CLI
Angular CLI	22.x (`npm install -g @angular/cli`)	Crear y ejecutar el proyecto Angular
Postman	Versión actual	Probar la API sin el front-end
Proyecto techstore-web	Del Taller 12 (opcional)	Si no lo tienes, el Paso 9 explica cómo crearlo

Conceptos que debes recordar

- Un **record** de Java es una clase inmutable y compacta: ideal para los DTO (objetos que viajan entre cliente y servidor).
- En Angular 22 los componentes son **standalone**: lo que usan se declara en `imports` del `@Component`.
- Los **signals** se leen con paréntesis (`clienteRegistrado()`) y se cambian con `set()` o `update()`.
- Códigos HTTP: **201** creado, **400** datos inválidos, **404** no existe, **409** conflicto con una regla de negocio, **500** error interno.

4. ¿Qué vas a construir?

Arquitectura del taller

```
[ Navegador ]                    [ API REST ]
techstore-web (Angular 22)       techstore-api (Spring Boot 4)
http://localhost:4200            http://localhost:8080

RegistroCliente  --- POST /api/clientes (JSON) --->  ClienteController (@Valid)
  Reactive Forms                                          |
  Validadores propios                                     v
  Mensajes de error  <--- 201 | 400 | 409 (JSON) ---  GlobalExceptionHandler
```

Proyecto	Archivo	Responsabilidad
techstore-api	dto/ClienteRequest.java	Datos de entrada + reglas de Bean Validation
techstore-api	dto/ClienteResponse.java · dto/ErrorResponse.java	Respuesta exitosa y formato único de error
techstore-api	exception/*.java	Excepciones propias y manejador global
techstore-api	service/ClienteService.java	Regla de negocio: DNI no repetido
techstore-api	controller/ClienteController.java	Endpoints POST y GET con @Valid
techstore-web	models/cliente.ts	Interfaces TypeScript de petición, respuesta y error
techstore-web	validators/peru-validators.ts	Validadores de DNI, celular, RUC y campos iguales
techstore-web	pages/registro-cliente/*	Formulario reactivo, plantilla y estilos
techstore-web	services/cliente-service.ts	Llamada HTTP a la API (Parte C)

5. Parte A: API con validación y manejo de errores (Spring Boot)

Paso 1. Crear el proyecto en Spring Initializr

Entra a **start.spring.io** (o en IntelliJ: File → New → Project → Spring Boot) y usa esta configuración:

Opción	Valor
Project / Language	Maven · Java
Spring Boot	La versión 4.x estable que ofrezca Initializr
Group / Artifact	pe.utp / techstore-api
Package name	pe.utp.techstore
Java	21
Dependencies	Spring Web · Validation

Descarga el ZIP, descomprímelo y ábrelo en IntelliJ. En el `pom.xml` deben aparecer estas dos dependencias (en Spring Boot 4 el iniciador web se llama `spring-boot-starter-webmvc`):

pom.xml (fragmento)

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

⚠**IMPORTANTE:** Si falta `spring-boot-starter-validation`, las anotaciones `@NotBlank`, `@Pattern`, etc. **no se ejecutan** y los datos inválidos entran sin error. Es el problema más frecuente del taller.

Paso 2. Crear la estructura de paquetes

Dentro de `src/main/java/pe/utp/techstore` crea los paquetes `controller`, `dto`, `exception` y `service`. La clase principal `TechstoreApiApplication` debe quedar en el paquete raíz `pe.utp.techstore` para que Spring encuentre todos los componentes.

Estructura

```
src/main/java/pe/utp/techstore/
├── TechstoreApiApplication.java
├── controller/ClienteController.java
├── dto/ClienteRequest.java · ClienteResponse.java · ErrorResponse.java
├── exception/DniDuplicadoException.java · RecursoNoEncontradoException.java
│             GlobalExceptionHandler.java
└── service/ClienteService.java
```

Paso 3. Crear los DTO con las reglas de validación

El DTO de entrada describe qué datos acepta la API y qué reglas deben cumplir. Cada anotación tiene un `message` en español: ese texto es el que verá el usuario en Angular.

dto/ClienteRequest.java

```
package pe.utp.techstore.dto;

import jakarta.validation.constraints.AssertTrue;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;

// DTO de entrada: lo que llega desde el formulario de Angular
public record ClienteRequest(

        @NotBlank(message = "Los nombres son obligatorios")
        @Size(min = 2, max = 50, message = "Los nombres deben tener entre 2 y 50 caracteres")
        String nombres,

        @NotBlank(message = "Los apellidos son obligatorios")
        @Size(min = 2, max = 50, message = "Los apellidos deben tener entre 2 y 50 caracteres")
        String apellidos,

        @NotBlank(message = "El DNI es obligatorio")
        @Pattern(regexp = "\\d{8}", message = "El DNI debe tener exactamente 8 dígitos")
        String dni,

        @NotBlank(message = "El correo es obligatorio")
        @Email(message = "El correo no tiene un formato válido")
        String email,

        @NotBlank(message = "El celular es obligatorio")
        @Pattern(regexp = "9\\d{8}", message = "El celular debe tener 9 dígitos y empezar con 9")
        String celular,

        @NotBlank(message = "El tipo de comprobante es obligatorio")
        @Pattern(regexp = "BOLETA|FACTURA", message = "El comprobante debe ser BOLETA o FACTURA")
        String tipoComprobante,

        String ruc,

        @NotBlank(message = "La contraseña es obligatoria")
        @Size(min = 8, message = "La contraseña debe tener al menos 8 caracteres")
        String password,

        @NotNull(message = "Debe aceptar los términos y condiciones")
        @AssertTrue(message = "Debe aceptar los términos y condiciones")
        Boolean aceptaTerminos
) {
    // Validación cruzada: si pide FACTURA, el RUC es obligatorio y debe ser válido
    @AssertTrue(message = "Para FACTURA el RUC es obligatorio: 11 dígitos que empiezan con 10 o
20")
    public boolean isRucValido() {
        if (!"FACTURA".equals(tipoComprobante)) {
            return true;
        }
        return ruc != null && ruc.matches("(10|20)\\d{9}");
    }
}
```

SABÍAS QUE: `@AssertTrue` sobre el método `isRucValido()` es una **validación cruzada**: revisa dos campos a la vez (tipo de comprobante y RUC). Como el método se llama `isRucValido`, el error aparecerá con la clave `rucValido`.

dto/ClienteResponse.java

```
package pe.utp.techstore.dto;

import java.time.LocalDateTime;

// DTO de salida: nunca devolvemos la contraseña
public record ClienteResponse(
        Long id,
        String nombres,
        String apellidos,
        String dni,
        String email,
        String celular,
        String tipoComprobante,
        String ruc,
        LocalDateTime fechaRegistro
) {
}
```

dto/ErrorResponse.java

```
package pe.utp.techstore.dto;

import java.time.LocalDateTime;
import java.util.Map;

// Formato único de error para TODA la API
public record ErrorResponse(
        LocalDateTime timestamp,
        int status,
        String error,
        String mensaje,
        String ruta,
        Map<String, String> errores
) {
}
```

Paso 4. Crear las excepciones propias

Una excepción propia representa una **regla de negocio**. No es un error de formato (eso lo ve Bean Validation), sino algo que solo el servidor sabe: por ejemplo, que el DNI ya está registrado.

exception/DniDuplicadoException.java

```
package pe.utp.techstore.exception;

// Regla de negocio: no pueden existir dos clientes con el mismo DNI
public class DniDuplicadoException extends RuntimeException {
    public DniDuplicadoException(String dni) {
        super("Ya existe un cliente registrado con el DNI " + dni);
    }
}
```

exception/RecursoNoEncontradoException.java

```
package pe.utp.techstore.exception;

public class RecursoNoEncontradoException extends RuntimeException {
    public RecursoNoEncontradoException(String mensaje) {
        super(mensaje);
    }
}
```

Paso 5. Crear el servicio

Para concentrarnos en la validación, el servicio guarda los clientes en memoria. En tu proyecto integrador lo reemplazarás por un repositorio JPA (sesión 6) y la contraseña se guardará cifrada con BCrypt (sesión 8).

service/ClienteService.java

```
package pe.utp.techstore.service;

import org.springframework.stereotype.Service;
import pe.utp.techstore.dto.ClienteRequest;
import pe.utp.techstore.dto.ClienteResponse;
import pe.utp.techstore.exception.DniDuplicadoException;
import pe.utp.techstore.exception.RecursoNoEncontradoException;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

// Servicio en memoria (en el proyecto final se reemplaza por un repositorio JPA)
@Service
public class ClienteService {

    private final Map<Long, ClienteResponse> clientes = new ConcurrentHashMap<>();
    private final AtomicLong secuencia = new AtomicLong(0);

    public ClienteResponse registrar(ClienteRequest req) {
        boolean dniExiste = clientes.values().stream()
                .anyMatch(c -> c.dni().equals(req.dni()));
        if (dniExiste) {
            throw new DniDuplicadoException(req.dni());
        }
        Long id = secuencia.incrementAndGet();
        String ruc = "FACTURA".equals(req.tipoComprobante()) ? req.ruc() : null;
        ClienteResponse nuevo = new ClienteResponse(id, req.nombres().trim(),
req.apellidos().trim(),
                req.dni(), req.email().toLowerCase(), req.celular(), req.tipoComprobante(), ruc,
                LocalDateTime.now());
        clientes.put(id, nuevo);
        return nuevo;
    }

    public List<ClienteResponse> listar() {
        return List.copyOf(clientes.values());
    }

    public ClienteResponse buscarPorId(Long id) {
        ClienteResponse cliente = clientes.get(id);
        if (cliente == null) {
            throw new RecursoNoEncontradoException("No existe el cliente con id " + id);
        }
        return cliente;
    }
}
```

Paso 6. Crear el controlador con @Valid

`@Valid` ordena a Spring ejecutar todas las reglas del DTO **antes** de entrar al método. Si alguna falla, el método no se ejecuta y Spring lanza `MethodArgumentNotValidException`. `@CrossOrigin` permite que Angular (puerto 4200) llame a la API (puerto 8080).

● controller/ClienteController.java

```
package pe.utp.techstore.controller;

import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import pe.utp.techstore.dto.ClienteRequest;
import pe.utp.techstore.dto.ClienteResponse;
import pe.utp.techstore.service.ClienteService;

import java.util.List;

@RestController
@RequestMapping("/api/clientes")
@CrossOrigin(origins = "http://localhost:4200") // permite llamadas desde Angular
public class ClienteController {

    private final ClienteService clienteService;

    public ClienteController(ClienteService clienteService) {
        this.clienteService = clienteService;
    }

    // @Valid activa Bean Validation ANTES de entrar al método
    @PostMapping
    public ResponseEntity<ClienteResponse> registrar(@Valid @RequestBody ClienteRequest request)
{
        ClienteResponse creado = clienteService.registrar(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(creado);
    }

    @GetMapping
    public List<ClienteResponse> listar() {
        return clienteService.listar();
    }

    @GetMapping("/{id}")
    public ClienteResponse buscar(@PathVariable Long id) {
        return clienteService.buscarPorId(id);
    }
}
```

Paso 7. Centralizar los errores con @RestControllerAdvice

Esta clase atrapa las excepciones de **todos** los controladores y las convierte en un JSON con el mismo formato. Cada método `@ExceptionHandler` decide el código HTTP y el mensaje. El último manejador (`Exception`) evita que el usuario vea trazas de Java: registra el detalle en el log del servidor y responde un mensaje amigable.

exception/GlobalExceptionHandler.java

```
package pe.utp.techstore.exception;

import jakarta.servlet.http.HttpServletRequest;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.servlet.resource.NoResourceFoundException;
import pe.utp.techstore.dto.ErrorResponse;

import java.time.LocalDateTime;
import java.util.LinkedHashMap;
import java.util.Map;

// Atrapa las excepciones de TODOS los controladores y responde un JSON uniforme
@RestControllerAdvice
public class GlobalExceptionHandler {

    private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);

    // 400: falló alguna anotación de Bean Validation (@NotBlank, @Pattern, ...)
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> manejarValidacion(MethodArgumentNotValidException ex,
                                                           HttpServletRequest request) {
        Map<String, String> errores = new LinkedHashMap<>();
        for (FieldError error : ex.getBindingResult().getFieldErrors()) {
            errores.putIfAbsent(error.getField(), error.getDefaultMessage());
        }
        return construir(HttpStatus.BAD_REQUEST, "Hay campos con datos inválidos", request,
errores);
    }

    // 400: el cuerpo no es un JSON válido (falta una llave, una coma, etc.)
    @ExceptionHandler(HttpMessageNotReadableException.class)
    public ResponseEntity<ErrorResponse> manejarJsonInvalido(HttpMessageNotReadableException ex,
                                                             HttpServletRequest request) {
        return construir(HttpStatus.BAD_REQUEST, "El cuerpo de la petición no es un JSON válido",
                request, Map.of());
    }

    // 409: regla de negocio (DNI repetido)
    @ExceptionHandler(DniDuplicadoException.class)
    public ResponseEntity<ErrorResponse> manejarDniDuplicado(DniDuplicadoException ex,
                                                             HttpServletRequest request) {
        return construir(HttpStatus.CONFLICT, "No se pudo completar el registro.", request,
                Map.of("dni", ex.getMessage()));
    }

    // 404: el recurso pedido no existe
    @ExceptionHandler(RecursoNoEncontradoException.class)
    public ResponseEntity<ErrorResponse> manejarNoEncontrado(RecursoNoEncontradoException ex,
                                                             HttpServletRequest request) {
        return construir(HttpStatus.NOT_FOUND, ex.getMessage(), request, Map.of());
    }

    // 404: la URL no corresponde a ningún endpoint
    @ExceptionHandler(NoResourceFoundException.class)
    public ResponseEntity<ErrorResponse> manejarRutaInexistente(NoResourceFoundException ex,
```

```
                                                                    HttpServletRequest request) {
        return construir(HttpStatus.NOT_FOUND, "La ruta solicitada no existe", request,
Map.of());
    }

    // 500: cualquier error no previsto. Se registra en el log, pero NO se muestra el detalle
técnico
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> manejarGeneral(Exception ex, HttpServletRequest request)
{
        log.error("Error no controlado en {}", request.getRequestURI(), ex);
        return construir(HttpStatus.INTERNAL_SERVER_ERROR,
                "Ocurrió un error interno. Inténtalo más tarde.", request, Map.of());
    }

    private ResponseEntity<ErrorResponse> construir(HttpStatus status, String mensaje,
                                                    HttpServletRequest request,
                                                    Map<String, String> errores) {
        ErrorResponse cuerpo = new ErrorResponse(LocalDateTime.now(), status.value(),
                status.getReasonPhrase(), mensaje, request.getRequestURI(), errores);
        return ResponseEntity.status(status).body(cuerpo);
    }
}
```

Excepción	Código HTTP	¿Cuándo ocurre?
MethodArgumentNotValidException	400 Bad Request	Falló una anotación de Bean Validation
HttpMessageNotReadableException	400 Bad Request	El cuerpo no es un JSON válido
DniDuplicadoException	409 Conflict	El DNI ya está registrado
RecursoNoEncontradoException	404 Not Found	GET de un id que no existe
NoResourceFoundException	404 Not Found	La URL no corresponde a ningún endpoint
Exception	500 Internal Server Error	Cualquier error no previsto

Paso 8. Ejecutar y probar en Postman

Ejecuta `TechstoreApiApplication` (botón ▶de IntelliJ). En la consola debe aparecer `Tomcat started on port 8080`. En Postman, para las peticiones POST usa **Body → raw → JSON** (no la pestaña Params). Este es un cuerpo válido:

```
● POST http://localhost:8080/api/clientes
```

```
{
  "nombres": "Ana",
  "apellidos": "Quispe Rojas",
  "dni": "70123456",
  "email": "ana.quispe@correo.pe",
  "celular": "987654321",
  "tipoComprobante": "BOLETA",
  "ruc": "",
  "password": "clave2026",
  "aceptaTerminos": true
}
```

Ejecuta los 8 casos de prueba y marca el resultado obtenido:

N.º	Petición	Cambio respecto al JSON válido	Esperado	Obtenido
1	POST /api/clientes	Ninguno	201 Created con id y fechaRegistro (sin password)	
2	POST /api/clientes	Repetir exactamente la petición 1	409 · errores.dni: "Ya existe un cliente…"	
3	POST /api/clientes	Enviar {} (cuerpo vacío)	400 con varios errores en "errores"	

N.º	Petición	Cambio respecto al JSON válido	Esperado	Obtenido
4	POST /api/clientes	"dni": "12AB", "celular": "812345678"	400 · errores.dni y errores.celular	
5	POST /api/clientes	"tipoComprobante": "FACTURA" sin RUC	400 · errores.rucValido	
6	POST /api/clientes	Borrar una coma del JSON	400 · "El cuerpo de la petición no es un JSON válido"	
7	GET /api/clientes/999	—	404 · "No existe el cliente con id 999"	
8	GET /api/clientez	URL mal escrita	404 · "La ruta solicitada no existe"	

● Respuesta esperada del caso 4 (400 Bad Request)

```
{
  "timestamp": "2026-11-04T19:35:12.481",
  "status": 400,
  "error": "Bad Request",
  "mensaje": "Hay campos con datos inválidos",
  "ruta": "/api/clientes",
  "errores": {
    "dni": "El DNI debe tener exactamente 8 dígitos",
    "celular": "El celular debe tener 9 dígitos y empezar con 9"
  }
}
```

✓**VERIFICA:** Los 8 casos responden con el código esperado.
Ninguna respuesta muestra trazas de Java ni la contraseña.
Deja la API ejecutándose: la usarás en la Parte C.

6. Parte B: formulario reactivo con validaciones (Angular)

Paso 9. Abrir o crear el proyecto techstore-web

Si tienes el proyecto del Taller 12, ábrelo en IntelliJ. Si no, créalo desde la terminal:

Terminal

```
ng new techstore-web --style=scss --ssr=false
cd techstore-web
ng serve
```

Si el asistente pregunta por herramientas de IA, elige **None**. Abre **http://localhost:4200** y deja `ng serve` ejecutándose: Angular recompila cada vez que guardas un archivo.

Si tu proyecto no tiene el archivo de variables SASS del Taller 12, créalo en `src/scss/_variables.scss`:

src/scss/_variables.scss

```
// Paleta institucional TechStore / UTP
$rojo-utp: #E30613;
$negro: #1A1A1A;
$gris-texto: #4D4D4D;
$gris-borde: #D0D5DD;
$gris-fondo: #F5F6F8;
$verde-ok: #1E8E3E;
$rojo-suave: #FDECEC;
$verde-suave: #E8F5EC;
```

Paso 10. Crear los modelos

Las interfaces describen la forma de los datos que viajan entre Angular y Spring Boot. Fíjate que coinciden campo por campo con los records de Java.

src/app/models/cliente.ts

```
// Datos que el formulario envía al back-end (Spring Boot)
export interface ClienteRequest {
  nombres: string;
  apellidos: string;
  dni: string;
  email: string;
  celular: string;
  tipoComprobante: 'BOLETA' | 'FACTURA';
  ruc: string;
  password: string;
  aceptaTerminos: boolean;
}

// Datos que el back-end devuelve cuando el registro es exitoso (201 Created)
export interface ClienteResponse {
  id: number;
  nombres: string;
  apellidos: string;
  dni: string;
  email: string;
  celular: string;
  tipoComprobante: string;
  ruc: string | null;
  fechaRegistro: string;
}

// Estructura del error que devuelve el @RestControllerAdvice
export interface ErrorResponse {
  timestamp: string;
  status: number;
  error: string;
  mensaje: string;
  ruta: string;
  errores: Record<string, string>;
}
```

Paso 11. Crear los validadores personalizados

Un validador es una función que recibe el control y devuelve `null` si el valor es correcto, o un objeto con la **clave del error** si es incorrecto. Si el campo está vacío devolvemos `null`, porque de eso se encarga `Validators.required`.

● src/app/validators/peru-validators.ts

```
import { AbstractControl, ValidationErrors, ValidatorFn } from '@angular/forms';

// 1) DNI peruano: exactamente 8 dígitos
export function dniValidator(control: AbstractControl): ValidationErrors | null {
  const valor = control.value as string;
  if (!valor) return null; // de los vacíos se encarga Validators.required
  return /^\d{8}$/.test(valor) ? null : { dniInvalido: true };
}

// 2) Celular peruano: 9 dígitos y empieza con 9
export function celularPeruValidator(control: AbstractControl): ValidationErrors | null {
  const valor = control.value as string;
  if (!valor) return null;
  return /^9\d{8}$/.test(valor) ? null : { celularInvalido: true };
}

// 3) RUC: 11 dígitos y empieza con 10 (persona natural) o 20 (empresa)
export function rucValidator(control: AbstractControl): ValidationErrors | null {
  const valor = control.value as string;
  if (!valor) return null;
  return /^(10|20)\d{9}$/.test(valor) ? null : { rucInvalido: true };
}

// 4) Validador cruzado (a nivel de grupo): dos campos deben ser iguales
export function camposIgualesValidator(campo: string, confirmacion: string): ValidatorFn {
  return (grupo: AbstractControl): ValidationErrors | null => {
    const valor1 = grupo.get(campo)?.value;
    const valor2 = grupo.get(confirmacion)?.value;
    if (!valor2) return null;
    return valor1 === valor2 ? null : { passwordsNoCoinciden: true };
  };
}
```

SABÍAS QUE: `camposIgualesValidator` es una **fábrica**: recibe los nombres de los campos y devuelve el validador. Así puedes reutilizarlo para confirmar un correo o una contraseña en cualquier formulario.

Paso 12. Generar el componente y crear el formulario

● Terminal

```
ng generate component pages/registro-cliente
```

Reemplaza el contenido de `registro-cliente.ts`. En esta Parte B el envío todavía **simula** la respuesta del servidor; en la Parte C lo conectarás a Spring Boot.

● src/app/pages/registro-cliente/registro-cliente.ts (versión Parte B)

```
import { Component, inject, signal } from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { NonNullableFormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import {
  camposIgualesValidator, celularPeruValidator, dniValidator, rucValidator,
} from '../../validators/peru-validators';
import { ClienteResponse } from '../../models/cliente';

@Component({
  selector: 'app-registro-cliente',
  imports: [ReactiveFormsModule],
  templateUrl: './registro-cliente.html',
```

```
  styleUrl: './registro-cliente.scss',
})
export class RegistroCliente {
  private readonly fb = inject(NonNullableFormBuilder);

  // 1. Modelo del formulario: cada campo con su valor inicial y sus validadores
  protected readonly form = this.fb.group(
    {
      nombres: ['', [Validators.required, Validators.minLength(2), Validators.maxLength(50)]],
      apellidos: ['', [Validators.required, Validators.minLength(2), Validators.maxLength(50)]],
      dni: ['', [Validators.required, dniValidator]],
      email: ['', [Validators.required, Validators.email]],
      celular: ['', [Validators.required, celularPeruValidator]],
      tipoComprobante: this.fb.control<'BOLETA' | 'FACTURA'>('BOLETA'),
      ruc: [''],
      password: ['', [Validators.required, Validators.minLength(8)]],
      confirmarPassword: ['', Validators.required],
      aceptaTerminos: [false, Validators.requiredTrue],
    },
    { validators: camposIgualesValidator('password', 'confirmarPassword') },
  );

  // 2. Estado de la pantalla con signals
  protected readonly enviando = signal(false);
  protected readonly clienteRegistrado = signal<ClienteResponse | null>(null);
  protected readonly mensajeGeneral = signal<string | null>(null);

  constructor() {
    // 3. Validación condicional: el RUC solo es obligatorio si el comprobante es FACTURA
    this.form.controls.tipoComprobante.valueChanges
      .pipe(takeUntilDestroyed())
      .subscribe((tipo) => {
        const ruc = this.form.controls.ruc;
        if (tipo === 'FACTURA') {
          ruc.setValidators([Validators.required, rucValidator]);
        } else {
          ruc.clearValidators();
          ruc.setValue('');
        }
        ruc.updateValueAndValidity();
      });
  }

  // 4. ¿Debo mostrar el error de este campo? Solo si es inválido y el usuario ya lo tocó
  protected mostrarError(campo: string): boolean {
    const control = this.form.get(campo);
    return !!control && control.invalid && control.touched;
  }

  // 5. Traduce el código de error a un mensaje claro para el usuario
  protected mensajeError(campo: string): string {
    const errores = this.form.get(campo)?.errors;
    if (!errores) return '';
    if (errores['servidor']) return errores['servidor'];
    if (errores['required']) {
      return campo === 'aceptaTerminos'
        ? 'Debes aceptar los términos y condiciones.'
        : 'Este campo es obligatorio.';
    }
    if (errores['minlength']) return `Debe tener al menos ${errores['minlength'].requiredLength}
caracteres.`;
    if (errores['maxlength']) return `Debe tener como máximo
${errores['maxlength'].requiredLength} caracteres.`;
    if (errores['email']) return 'Ingresa un correo válido, por ejemplo: ana@correo.com';
    if (errores['dniInvalido']) return 'El DNI debe tener exactamente 8 dígitos.';
    if (errores['celularInvalido']) return 'El celular debe tener 9 dígitos y empezar con 9.';
    if (errores['rucInvalido']) return 'El RUC debe tener 11 dígitos y empezar con 10 o 20.';
    return 'El valor ingresado no es válido.';
  }

  // 6. Error del grupo (validador cruzado)
  protected passwordsNoCoinciden(): boolean {
    return this.form.hasError('passwordsNoCoinciden') &&
this.form.controls.confirmarPassword.touched;
  }
```

```
  // 7. Envío del formulario (Parte B: todavía sin back-end)
  protected registrar(): void {
    this.mensajeGeneral.set(null);
    if (this.form.invalid) {
      this.form.markAllAsTouched(); // muestra todos los errores de una vez
      return;
    }
    const { confirmarPassword, password, aceptaTerminos, ...cliente } = this.form.getRawValue();
    // Simulamos la respuesta del servidor; en la Parte C la enviará Spring Boot
    this.clienteRegistrado.set({ id: 1, ...cliente, fechaRegistro: new Date().toISOString() });
    this.form.reset();
  }

  protected nuevoRegistro(): void {
    this.clienteRegistrado.set(null);
  }
}
```

Bloque	¿Qué hace?
1. Modelo del formulario	Cada campo: [valor inicial, validadores]. El segundo argumento de group() registra el validador cruzado.
2. Estado con signals	enviando, clienteRegistrado y mensajeGeneral actualizan la vista automáticamente.
3. Validación condicional	Escucha valueChanges del tipo de comprobante y activa o quita las reglas del RUC.
4. mostrarError()	Regla de oro: el error se ve solo si el campo es inválido y ya fue tocado.
5. mensajeError()	Traduce la clave del error a un mensaje claro. Todos los campos usan la misma función.
6. passwordsNoCoinciden()	Lee el error del grupo (no de un control).
7. registrar()	Si hay errores llama a markAllAsTouched() para mostrarlos todos; si no, registra.

Paso 13. Crear la plantilla

`[formGroup]` une el formulario con la clase; `formControlName` une cada campo; `(ngSubmit)` llama a `registrar()`. El atributo `novalidate` desactiva los globos de validación del navegador para usar nuestros propios mensajes.

● src/app/pages/registro-cliente/registro-cliente.html

```
<section class="tarjeta">
  <h1>Registro de cliente</h1>
  <p class="subtitulo">Crea tu cuenta en TechStore Perú para comprar con boleta o factura.</p>

  @if (clienteRegistrado(); as cliente) {
    <div class="exito">
      <h2>¡Registro exitoso!</h2>
      <p>Bienvenido(a), <strong>{{ cliente.nombres }} {{ cliente.apellidos }}</strong>.
        Tu código de cliente es <strong>#{{ cliente.id }}</strong>.</p>
      <button type="button" class="btn" (click)="nuevoRegistro()">Registrar otro cliente</button>
    </div>
  } @else {
    @if (mensajeGeneral()) {
      <div class="alerta" role="alert">{{ mensajeGeneral() }}</div>
    }

    <form [formGroup]="form" (ngSubmit)="registrar()" novalidate>
      <div class="fila">
        <div class="campo">
          <label for="nombres">Nombres *</label>
          <input id="nombres" formControlName="nombres"
[class.invalido]="mostrarError('nombres')" />
          @if (mostrarError('nombres')) { <small class="error">{{ mensajeError('nombres')
}}</small> }
```

```
        </div>
        <div class="campo">
          <label for="apellidos">Apellidos *</label>
          <input id="apellidos" formControlName="apellidos"
[class.invalido]="mostrarError('apellidos')" />
          @if (mostrarError('apellidos')) { <small class="error">{{ mensajeError('apellidos')
}}</small> }
        </div>
      </div>

      <div class="fila">
        <div class="campo">
          <label for="dni">DNI *</label>
          <input id="dni" formControlName="dni" maxlength="8" inputmode="numeric"
[class.invalido]="mostrarError('dni')" />
          @if (mostrarError('dni')) { <small class="error">{{ mensajeError('dni') }}</small> }
        </div>
        <div class="campo">
          <label for="celular">Celular *</label>
          <input id="celular" formControlName="celular" maxlength="9" inputmode="numeric"
[class.invalido]="mostrarError('celular')" />
          @if (mostrarError('celular')) { <small class="error">{{ mensajeError('celular')
}}</small> }
        </div>
      </div>

      <div class="campo">
        <label for="email">Correo electrónico *</label>
        <input id="email" type="email" formControlName="email"
[class.invalido]="mostrarError('email')" />
        @if (mostrarError('email')) { <small class="error">{{ mensajeError('email') }}</small> }
      </div>

      <div class="fila">
        <div class="campo">
          <label for="tipoComprobante">Tipo de comprobante</label>
          <select id="tipoComprobante" formControlName="tipoComprobante">
            <option value="BOLETA">Boleta</option>
            <option value="FACTURA">Factura</option>
          </select>
        </div>
        @if (form.controls.tipoComprobante.value === 'FACTURA') {
          <div class="campo">
            <label for="ruc">RUC *</label>
            <input id="ruc" formControlName="ruc" maxlength="11" inputmode="numeric"
[class.invalido]="mostrarError('ruc')" />
            @if (mostrarError('ruc')) { <small class="error">{{ mensajeError('ruc') }}</small> }
          </div>
        }
      </div>

      <div class="fila">
        <div class="campo">
          <label for="password">Contraseña *</label>
          <input id="password" type="password" formControlName="password"
[class.invalido]="mostrarError('password')" />
          @if (mostrarError('password')) { <small class="error">{{ mensajeError('password')
}}</small> }
        </div>
        <div class="campo">
          <label for="confirmarPassword">Confirmar contraseña *</label>
          <input id="confirmarPassword" type="password" formControlName="confirmarPassword"
                 [class.invalido]="mostrarError('confirmarPassword') || passwordsNoCoinciden()"
/>
          @if (mostrarError('confirmarPassword')) {
            <small class="error">{{ mensajeError('confirmarPassword') }}</small>
          } @else if (passwordsNoCoinciden()) {
            <small class="error">Las contraseñas no coinciden.</small>
          }
        </div>
      </div>

      <div class="campo check">
        <label>
          <input type="checkbox" formControlName="aceptaTerminos" />
          Acepto los términos y condiciones
```

```
        </label>
        @if (mostrarError('aceptaTerminos')) { <small class="error">{{
mensajeError('aceptaTerminos') }}</small> }
      </div>

      <button type="submit" class="btn" [disabled]="enviando()">
        {{ enviando() ? 'Registrando...' : 'Registrar' }}
      </button>
    </form>

    <!-- Panel didáctico: estado del formulario en tiempo real -->
    <aside class="estado">
      <span>valid: <b>{{ form.valid }}</b></span>
      <span>touched: <b>{{ form.touched }}</b></span>
      <span>dirty: <b>{{ form.dirty }}</b></span>
    </aside>
  }
</section>
```

Paso 14. Agregar los estilos

src/app/pages/registro-cliente/registro-cliente.scss

```
@use '../../../scss/variables' as v;

.tarjeta {
  max-width: 720px;
  margin: 32px auto;
  background: #fff;
  border-radius: 12px;
  padding: 28px 32px;
  box-shadow: 0 4px 16px rgba(0, 0, 0, 0.08);
}
h1 { margin: 0; color: v.$rojo-utp; }
.subtitulo { color: v.$gris-texto; margin: 4px 0 20px; }
.fila { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
.campo {
  display: flex; flex-direction: column; margin-bottom: 14px;
  label { font-weight: 600; margin-bottom: 4px; font-size: 14px; }
  input, select {
    padding: 9px 11px; border: 1px solid v.$gris-borde; border-radius: 8px; font-size: 15px;
    &:focus { outline: 2px solid rgba(v.$rojo-utp, 0.25); border-color: v.$rojo-utp; }
  }
  input.invalido { border-color: v.$rojo-utp; background: v.$rojo-suave; }
}
.check label { font-weight: 400; display: flex; gap: 8px; align-items: center; }
.error { color: v.$rojo-utp; font-size: 13px; margin-top: 4px; }
.btn {
  background: v.$rojo-utp; color: #fff; border: 0; border-radius: 8px;
  padding: 11px 22px; font-size: 15px; font-weight: 600; cursor: pointer;
  &:disabled { opacity: 0.6; cursor: wait; }
}
.alerta {
  background: v.$rojo-suave; color: #8A0410; border: 1px solid v.$rojo-utp;
  border-radius: 8px; padding: 10px 14px; margin-bottom: 16px;
}
.exito {
  background: v.$verde-suave; border: 1px solid v.$verde-ok; border-radius: 10px; padding: 18px;
  h2 { color: v.$verde-ok; margin-top: 0; }
}
.estado {
  margin-top: 18px; display: flex; gap: 18px; font-family: monospace; font-size: 13px;
  background: v.$gris-fondo; padding: 8px 12px; border-radius: 8px; color: v.$gris-texto;
}
```

Paso 15. Registrar la ruta

Agrega la ruta `registro` en `app.routes.ts` (antes de la ruta comodín `**` si la tienes) y un enlace en el menú del encabezado:

● src/app/app.routes.ts (fragmento)

```
import { RegistroCliente } from './pages/registro-cliente/registro-cliente';

export const routes: Routes = [
  // ... tus rutas del Taller 12
  { path: 'registro', component: RegistroCliente, title: 'TechStore | Registro de cliente' },
  // { path: '**', component: NoEncontrado },  <- la ruta comodín siempre al final
];
```

● Enlace en el encabezado (header.html o app.html)

```
<a routerLink="/registro">Registro</a>
```

Abre **http://localhost:4200/registro** y ejecuta estas pruebas:

N.º	Acción	Resultado esperado	✓
1	Presionar Registrar con el formulario vacío	Aparecen 8 mensajes de error a la vez	
2	Escribir "12AB" en DNI y salir del campo	El DNI debe tener exactamente 8 dígitos	
3	Escribir "812345678" en Celular	El celular debe tener 9 dígitos y empezar con 9	
4	Escribir "ana@" en Correo	Ingresa un correo válido…	
5	Elegir Factura	Aparece el campo RUC (obligatorio)	
6	Escribir "123" en RUC	El RUC debe tener 11 dígitos y empezar con 10 o 20	
7	Volver a Boleta	El campo RUC desaparece y deja de ser obligatorio	
8	Contraseñas distintas	Las contraseñas no coinciden	
9	Llenar todo correctamente	valid: true en el panel y mensaje de registro exitoso	



Prueba 1: todos los errores al enviar el formulario vacío

✓**VERIFICA:** Las 9 pruebas se cumplen.
El panel inferior muestra valid, touched y dirty cambiando en tiempo real.

7. Parte C: mostrar en el formulario los errores del servidor

Ahora conectaremos el formulario con la API. Esta parte es un adelanto de la sesión 14, donde estudiarás a fondo `HttpClient`. Lo importante hoy es el **manejo de errores**: cada mensaje que devuelve Spring Boot debe aparecer debajo de su campo.

Paso 16. Habilitar HttpClient

Verifica que `app.config.ts` incluya `provideHttpClient()`:

src/app/app.config.ts

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes, withComponentInputBinding()),
    provideHttpClient(),
  ],
};
```

Paso 17. Crear el servicio de clientes

Terminal

```
ng generate service services/cliente-service
```

src/app/services/cliente-service.ts

```
import { HttpClient } from '@angular/common/http';
import { Injectable, inject } from '@angular/core';
import { Observable } from 'rxjs';
import { ClienteRequest, ClienteResponse } from '../models/cliente';

@Injectable({ providedIn: 'root' })
export class ClienteService {
  private readonly http = inject(HttpClient);
  private readonly apiUrl = 'http://localhost:8080/api/clientes';

  registrar(cliente: ClienteRequest): Observable<ClienteResponse> {
    return this.http.post<ClienteResponse>(this.apiUrl, cliente);
  }
}
```

Paso 18. Versión final del componente

Reemplaza `registro-cliente.ts` por esta versión. Los cambios respecto a la Parte B están en los imports, en la inyección de `ClienteService`, en `registrar()` (bloque 7) y en el nuevo método `mostrarErroresDelServidor()` (bloque 8).

src/app/pages/registro-cliente/registro-cliente.ts (versión final)

```
import { Component, inject, signal } from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { HttpErrorResponse } from '@angular/common/http';
import { NonNullableFormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import {
  camposIgualesValidator, celularPeruValidator, dniValidator, rucValidator,
} from '../../validators/peru-validators';
import { ClienteRequest, ClienteResponse, ErrorResponse } from '../../models/cliente';
import { ClienteService } from '../../services/cliente-service';

@Component({
```

```
  selector: 'app-registro-cliente',
  imports: [ReactiveFormsModule],
  templateUrl: './registro-cliente.html',
  styleUrl: './registro-cliente.scss',
})
export class RegistroCliente {
  private readonly fb = inject(NonNullableFormBuilder);
  private readonly clienteService = inject(ClienteService);

  // 1. Modelo del formulario: cada campo con su valor inicial y sus validadores
  protected readonly form = this.fb.group(
    {
      nombres: ['', [Validators.required, Validators.minLength(2), Validators.maxLength(50)]],
      apellidos: ['', [Validators.required, Validators.minLength(2), Validators.maxLength(50)]],
      dni: ['', [Validators.required, dniValidator]],
      email: ['', [Validators.required, Validators.email]],
      celular: ['', [Validators.required, celularPeruValidator]],
      tipoComprobante: this.fb.control<'BOLETA' | 'FACTURA'>('BOLETA'),
      ruc: [''],
      password: ['', [Validators.required, Validators.minLength(8)]],
      confirmarPassword: ['', Validators.required],
      aceptaTerminos: [false, Validators.requiredTrue],
    },
    { validators: camposIgualesValidator('password', 'confirmarPassword') },
  );

  // 2. Estado de la pantalla con signals
  protected readonly enviando = signal(false);
  protected readonly clienteRegistrado = signal<ClienteResponse | null>(null);
  protected readonly mensajeGeneral = signal<string | null>(null);

  constructor() {
    // 3. Validación condicional: el RUC solo es obligatorio si el comprobante es FACTURA
    this.form.controls.tipoComprobante.valueChanges
      .pipe(takeUntilDestroyed())
      .subscribe((tipo) => {
        const ruc = this.form.controls.ruc;
        if (tipo === 'FACTURA') {
          ruc.setValidators([Validators.required, rucValidator]);
        } else {
          ruc.clearValidators();
          ruc.setValue('');
        }
        ruc.updateValueAndValidity();
      });
  }

  // 4. ¿Debo mostrar el error de este campo? Solo si es inválido y el usuario ya lo tocó
  protected mostrarError(campo: string): boolean {
    const control = this.form.get(campo);
    return !!control && control.invalid && control.touched;
  }

  // 5. Traduce el código de error a un mensaje claro para el usuario
  protected mensajeError(campo: string): string {
    const errores = this.form.get(campo)?.errors;
    if (!errores) return '';
    if (errores['servidor']) return errores['servidor'];
    if (errores['required']) {
      return campo === 'aceptaTerminos'
        ? 'Debes aceptar los términos y condiciones.'
        : 'Este campo es obligatorio.';
    }
    if (errores['minlength']) return `Debe tener al menos ${errores['minlength'].requiredLength}
caracteres.`;
    if (errores['maxlength']) return `Debe tener como máximo
${errores['maxlength'].requiredLength} caracteres.`;
    if (errores['email']) return 'Ingresa un correo válido, por ejemplo: ana@correo.com';
    if (errores['dniInvalido']) return 'El DNI debe tener exactamente 8 dígitos.';
    if (errores['celularInvalido']) return 'El celular debe tener 9 dígitos y empezar con 9.';
    if (errores['rucInvalido']) return 'El RUC debe tener 11 dígitos y empezar con 10 o 20.';
    return 'El valor ingresado no es válido.';
  }

  // 6. Error del grupo (validador cruzado)
  protected passwordsNoCoinciden(): boolean {
```

```
    return this.form.hasError('passwordsNoCoinciden') &&
this.form.controls.confirmarPassword.touched;
  }

  // 7. Envío del formulario
  protected registrar(): void {
    this.mensajeGeneral.set(null);
    if (this.form.invalid) {
      this.form.markAllAsTouched(); // muestra todos los errores de una vez
      return;
    }
    const { confirmarPassword, ...datos } = this.form.getRawValue();
    this.enviando.set(true);
    this.clienteService.registrar(datos as ClienteRequest).subscribe({
      next: (cliente) => {
        this.enviando.set(false);
        this.clienteRegistrado.set(cliente);
        this.form.reset();
      },
      error: (err: HttpErrorResponse) => {
        this.enviando.set(false);
        this.mostrarErroresDelServidor(err);
      },
    });
  }

  // 8. Manejo de errores HTTP: coloca cada mensaje del back-end en su campo
  private mostrarErroresDelServidor(err: HttpErrorResponse): void {
    if (err.status === 0) {
      this.mensajeGeneral.set('No se pudo conectar con el servidor. Verifica que la API esté
encendida.');
      return;
    }
    const cuerpo = err.error as ErrorResponse | null;
    const errores = cuerpo?.errores ?? {};
    const sinCampo: string[] = [];
    for (const [campo, mensaje] of Object.entries(errores)) {
      const control = this.form.get(campo);
      if (control) {
        control.setErrors({ servidor: mensaje });
        control.markAsTouched();
      } else {
        sinCampo.push(mensaje);
      }
    }
    this.mensajeGeneral.set(
      [cuerpo?.mensaje ?? 'Ocurrió un error inesperado. Inténtalo nuevamente.',
...sinCampo].join(' '),
    );
  }

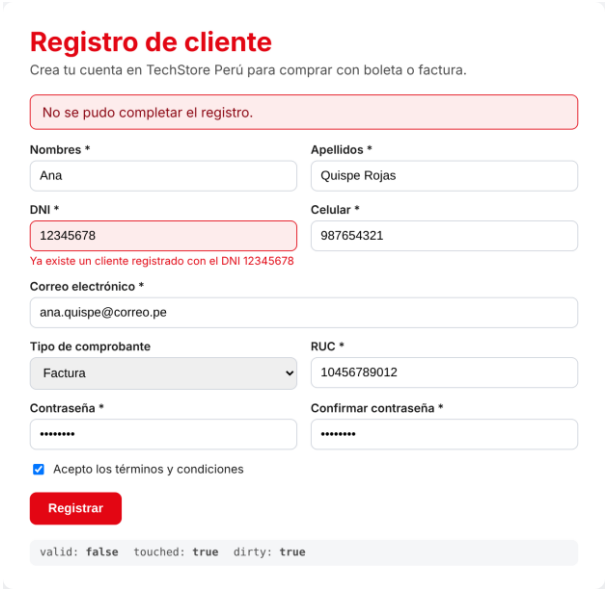
  protected nuevoRegistro(): void {
    this.clienteRegistrado.set(null);
  }
}
```

Situación	err.status	¿Qué hace el componente?
La API está apagada o hay un problema de CORS	0	Muestra "No se pudo conectar con el servidor…"
Datos inválidos según Bean Validation	400	Coloca cada mensaje de errores en su control con setErrors({ servidor })
DNI ya registrado	409	Muestra el mensaje bajo el campo DNI y la alerta general
Clave sin campo en el formulario (ej. rucValido)	400	Agrega el mensaje a la alerta general

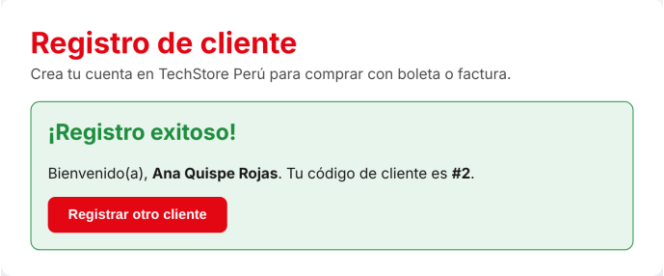
Paso 19. Probar la integración

Con la API (Parte A) y Angular ejecutándose a la vez:

N.º	Acción	Resultado esperado	✓
1	Registrar a un cliente con datos válidos	Mensaje "¡Registro exitoso!" con el código de cliente	
2	Registrar otro cliente con el mismo DNI	Bajo el campo DNI: "Ya existe un cliente registrado con el DNI…"	
3	Corregir el DNI y volver a registrar	El error desaparece al escribir y el registro es exitoso	
4	Detener la API y registrar	Alerta: "No se pudo conectar con el servidor…"	



Prueba 2: el error 409 del back-end aparece bajo el campo DNI



Prueba 1: registro exitoso (201 Created)

✓**VERIFICA:** Las 4 pruebas se cumplen.
En DevTools (F12 → Network) ves el POST con status 201, 409 o 400 según el caso.

8. Errores frecuentes y cómo resolverlos

Síntoma	Causa	Solución
Can't bind to 'formGroup' since it isn't a known property of 'form'	Falta importar ReactiveFormsModule	Agregar ReactiveFormsModule en imports del @Component
NullInjectorError: No provider for HttpClient	Falta provideHttpClient()	Agregarlo en providers de app.config.ts
Los errores no aparecen al presionar Registrar	No se llama a markAllAsTouched()	Llamarlo cuando form.invalid sea true

Síntoma	Causa	Solución
El back-end acepta datos inválidos	Falta @Valid o la dependencia Validation	Agregar @Valid antes de @RequestBody y spring-boot-starter-validation
Error de CORS en la consola del navegador	La API no permite el origen 4200	Revisar @CrossOrigin(origins = "http://localhost:4200")
415 Unsupported Media Type en Postman	El cuerpo no se envió como JSON	Body → raw → JSON
err.status es 0	API apagada, puerto distinto o CORS	Verificar que la API esté en el puerto 8080
Port 8080 was already in use	Otra ejecución de la API sigue abierta	Detener la ejecución anterior (■) en IntelliJ
El RUC sigue siendo obligatorio con Boleta	Falta updateValueAndValidity()	Llamarlo después de cambiar los validadores

9. Retos adicionales

Reto 1 (inicial): suscripción al boletín

Crea el componente `Boletin` con nombre (obligatorio, mínimo 3 letras) y correo (obligatorio, formato válido). El botón debe estar deshabilitado mientras el formulario sea inválido y, al enviar, mostrar "¡Gracias, {nombre}!" usando un signal.

Reto 2 (intermedio): nuevo producto

Crea el componente `NuevoProducto` con nombre, categoría (select obligatorio), precio (mínimo 1), stock (entero, mínimo 0) y precio de oferta opcional. Crea un validador de grupo que exija que la oferta sea menor que el precio. Al guardar, agrega el producto a una lista con `signal.update()`.

Reto 3 (avanzado): API de productos con errores controlados

En techstore-api crea `ProductoRequest` con `@NotBlank`, `@Pattern(regexp = "TS-\\d{4}")`, `@Positive` y `@PositiveOrZero`; el endpoint `POST /api/productos` (201) y la excepción `CodigoDuplicadoException`, que el `GlobalExceptionHandler` debe convertir en 409. Pruébalo en Postman con un caso válido, uno inválido y uno duplicado.

Reto 4 (opcional): Signal Forms

Angular 22 declaró estables los **Signal Forms**. Reescribe el Reto 1 con `form()`, `FormField` y las reglas `required()` y `email()` de `@angular/forms/signals`, y compara cuántas líneas necesitaste en cada versión.

10. Preguntas de reflexión

1. ¿Por qué la API debe validar los datos aunque Angular ya los haya validado? Da un ejemplo con Postman.
2. ¿Qué diferencia hay entre un error de formato (400) y un error de regla de negocio (409)? ¿Quién detecta cada uno?
3. ¿Por qué el manejador de `Exception` no debe devolver el mensaje técnico de la excepción al usuario?
4. ¿Qué pasaría si `mostrarError()` no revisara `touched`? ¿Cómo afectaría la experiencia del usuario?
5. El error de la validación cruzada del RUC llega con la clave `rucValido`. ¿Qué cambiarías para que aparezca bajo el campo RUC?

11. Entregables

- Proyectos **techstore-api** y **techstore-web** comprimidos en un ZIP (sin las carpetas `node_modules` ni `target`).
- Capturas de Postman de los casos 1, 2, 4 y 5 de la Parte A.
- Capturas del formulario: errores al enviar vacío, error 409 bajo el DNI y registro exitoso.
- Respuestas a las preguntas de reflexión (máximo una página).
- Publica en el foro del curso una captura de tu formulario funcionando y una dificultad que resolviste.